ITSOLERA

SOC Analyst Internship Program

Internship Task Booklet

Compiled Task Documentation

Prepared by:
Abdullah Khan
SOC Analyst Intern

2025 – 2026

Task 01 File Integrity Monitoring with Wazuh

1. Introduction & Project Scope

The primary objective of this task was to integrate a Kali Linux agent with the Wazuh Manager and configure File Integrity Monitoring (FIM) for real-time threat detection. This setup enables automated identification of unauthorized file creations, modifications, and deletions within critical system directories. Successfully implementing this architecture laid the groundwork for advanced log analysis and incident response capabilities in a controlled lab environment.

2. Importance of File Integrity Monitoring

FIM plays a crucial role in maintaining the CIA Triad — particularly the Integrity pillar. It ensures critical system files and configurations are protected from unauthorized changes, helping to prevent file tampering and malicious modifications. Additionally, FIM detects suspicious file activities such as ransomware encryption or malicious script drops, and identifies threats that bypass traditional perimeter security controls. It also maintains detailed logs of file changes — including cryptographic hashes, permissions, and ownership — which help analysts reconstruct attack timelines and assess impact.

3. Environment Setup

The lab was hosted using virtualization technology to isolate the security infrastructure. The configuration was as follows:

- Host Machine: Windows Workstation
- Virtualization Platform: Oracle VirtualBox
- VM 1 — Wazuh Manager (OVA): 4 GB RAM, 2 vCPUs, NAT Network with Port Forwarding
- VM 2 — Wazuh Agent (Kali Linux): 2 GB RAM, 2 CPUs, NAT Network
- Tools Used: Oracle VirtualBox, Wazuh Platform v4.14, Kali Linux

4. Phase 1: Installation & Configuration

The Wazuh OVA file was downloaded from the official documentation portal. It bundles three core components: the Wazuh Indexer (stores and indexes security alerts), the Wazuh Server (processes agent data and generates alerts based on defined rules), and the Wazuh Dashboard (web interface for visualization and agent management). After importing the OVA into VirtualBox, the VM's network adapter was configured to NAT with port forwarding, allowing secure access to the Wazuh Dashboard from the host browser at https://127.0.0.1:8443.

The Wazuh Agent was deployed on Kali Linux by following the Add Agent wizard on the dashboard. The generated installation command was executed in the Linux terminal, and the agent was started and enabled using:

```
sudo systemctl start wazuh-agent
sudo systemctl enable wazuh-agent
```

Successful connection was verified by returning to the dashboard, where the Kali agent appeared as active.

5. Phase 2: Implementing FIM

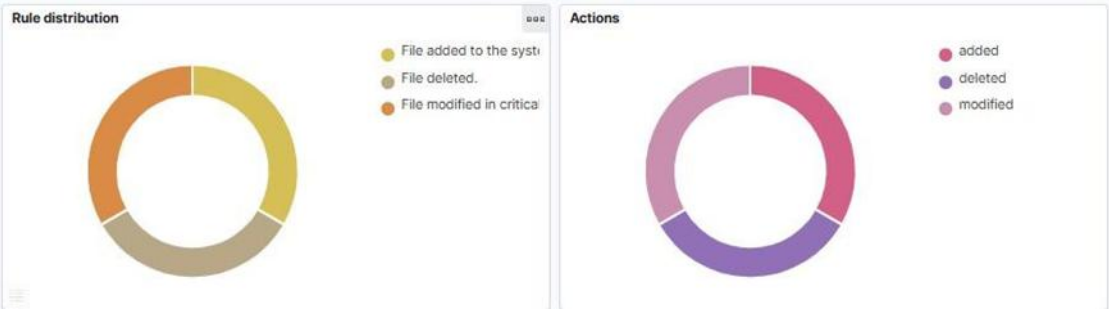
The ossec.conf file located at /var/ossec/etc/ossec.conf is the main configuration file for the Wazuh agent. The Syscheck module within this file is responsible for File Integrity Monitoring. It was edited to define the monitoring path:

```
<directories check_all="yes" realtime="yes">/home/kali/fim_testing</directories>
```

A custom FIM rule was added on the Wazuh Manager to generate a High Severity alert (Level 10) whenever any change occurred within the monitored folder. Services were restarted on both the agent and manager to apply all changes.

6. Phase 3: Testing & Validation

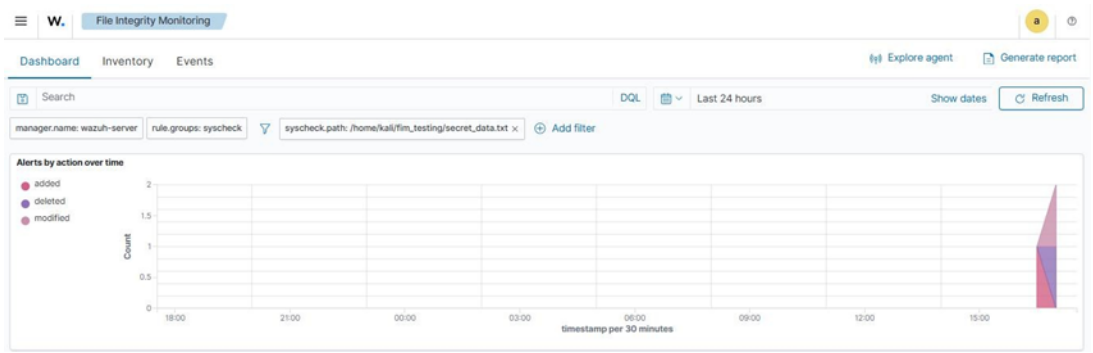
A test directory was created at /home/kali/fim_testing. Files were created, modified, and deleted using terminal commands. Each action triggered an alert on the Wazuh Dashboard, confirmed under the Integrity Monitoring section. The dashboard clearly displayed file creation, modification, and deletion events — each with timestamps, file paths, and change details.

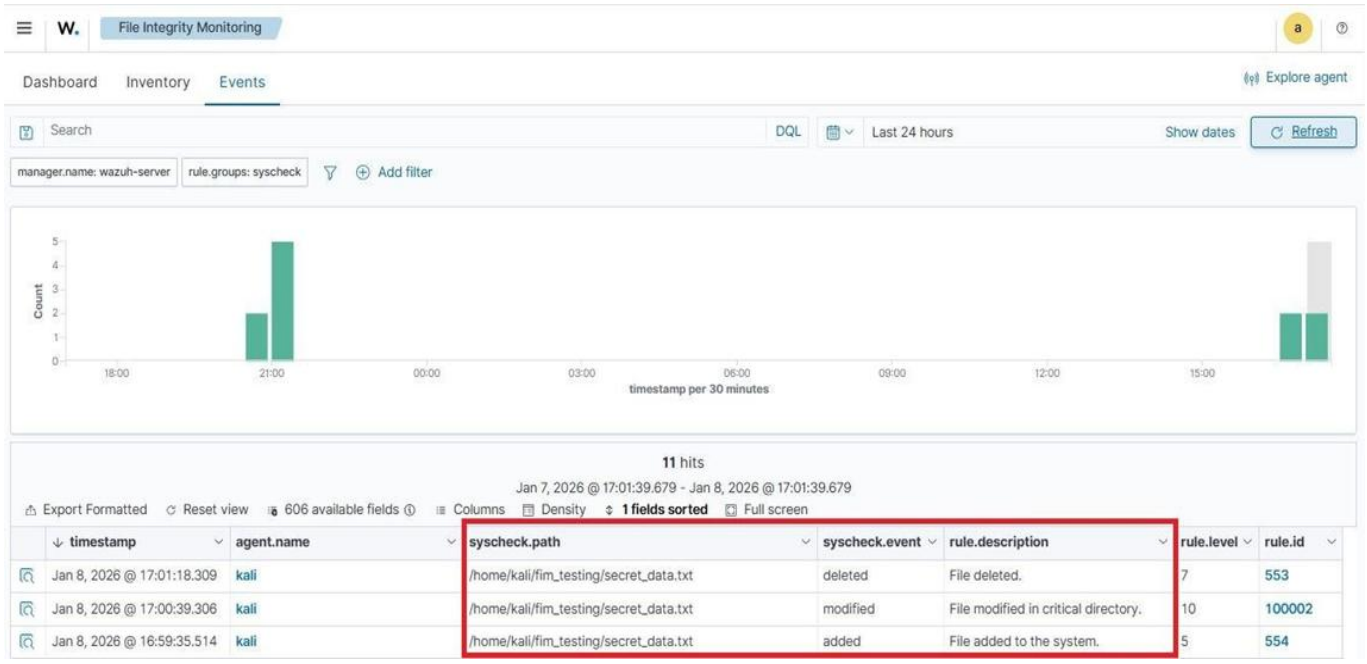


7. Phase 4: Challenges & Conclusion

Several challenges were encountered and resolved during this task. XML syntax errors in custom rules initially prevented the Wazuh Manager from restarting — these were fixed by inspecting ossec.log. A time synchronization issue between the Kali agent and the Wazuh Server caused alerts to not appear immediately; this was resolved by adjusting the dashboard time filter to 'Last 24 Hours'. Permission denied errors were handled by prefixing commands with sudo.

In conclusion, FIM was successfully configured on /home/kali/fim_testing. The custom rule (ID: 100002) generated High-Severity alerts at Level 10 for any file change. This lab provided hands-on experience with Linux terminal operations, Wazuh XML configuration, log troubleshooting, and security monitoring fundamentals.





Task 02 pfSense Firewall Integration with Wazuh

1. Introduction

pfSense is an open-source firewall and router built on FreeBSD, providing advanced traffic filtering and network security features. Wazuh is an open-source security monitoring platform used for log analysis and threat detection. Integrating pfSense with Wazuh enables centralized monitoring of firewall logs and security events, allowing real-time visibility into allowed and blocked network traffic through a unified dashboard, improving incident detection and response.

2. Learning Outcomes

1. Install and configure pfSense on VirtualBox to establish a functional firewall.
2. Create firewall rules to block access to specific IP addresses (Google and Facebook).
3. Verify the effectiveness of these rules by testing from Kali Linux.
4. Integrate pfSense with Wazuh for real-time monitoring and logging of firewall events.
5. Analyze and visualize security events related to firewall activities for improved threat detection.

3. Network Architecture

IP addresses were assigned and verified across all virtual machines:

- Kali Linux: 192.168.100.25
- Wazuh Server: 192.168.100.50
- pfSense: 192.168.100.100

4. pfSense Installation & Setup

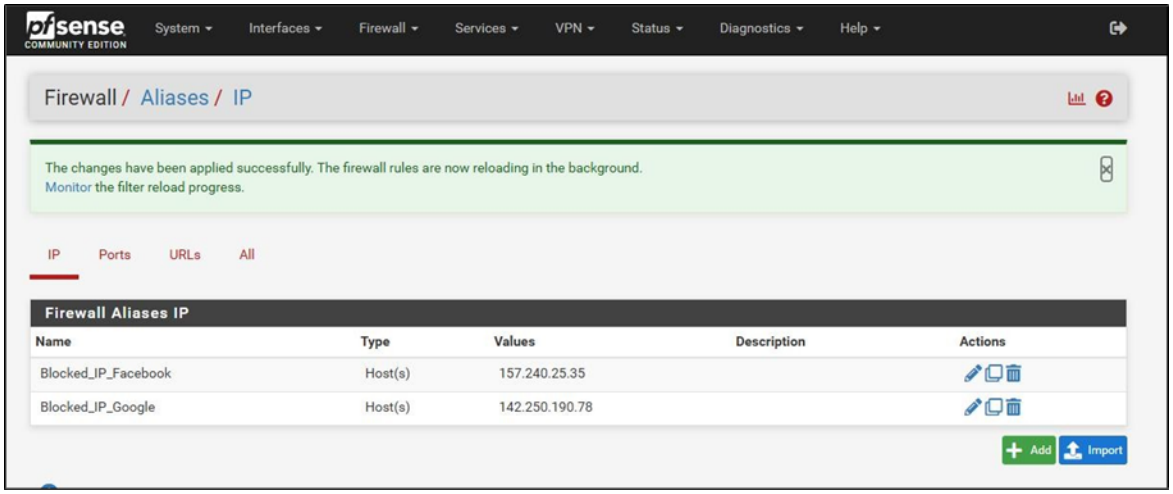
pfSense Community Edition (v2.7.2) was downloaded and imported into VirtualBox. The VM was configured with 4096 MB RAM, 2 CPUs, and 25 GB disk. Network adapters were configured as Bridged for pfSense (to connect to the host network) and Bridged for Wazuh, while Kali was placed on an Internal Network so it communicates only through pfSense.

During installation, the Auto (FS) Guided Root-on-ZFS partitioning was selected for data safety. After installation, the ISO was removed and the VM rebooted. The WAN interface was manually assigned IP 192.168.100.100/24 using the console menu. The web interface was then accessed at this IP using credentials admin/pfsense, and the Setup Wizard was completed — including unchecking 'Block RFC1918 Private Networks' to allow internal lab traffic to flow correctly.

5. Firewall Configuration & Testing

Two IP aliases were created under Firewall > Aliases: one for Google (142.250.190.78) and one for Facebook (157.240.25.35). Firewall rules were then created to block traffic from Kali's IP to each alias, with packet logging enabled. These rules were applied to the WAN interface, and Kali's gateway was set to pfSense (192.168.100.100) to ensure all traffic routes through the firewall.

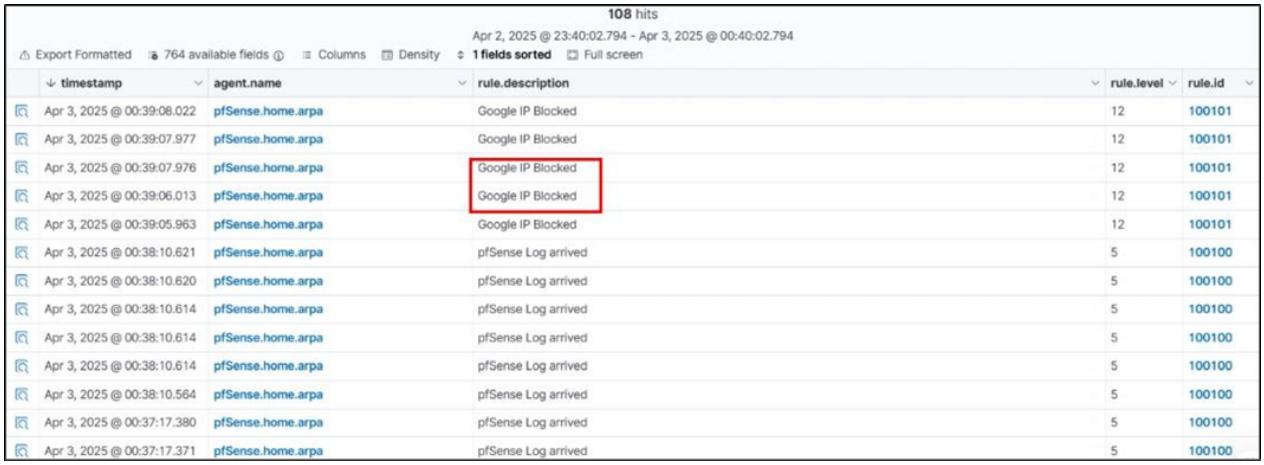
Verification was performed from the Kali Linux terminal using ping commands against both blocked IPs — both returned no reply, confirming the rules were working. pfSense firewall logs under Status > System Logs > Firewall showed the blocked packets from Kali.



6. Wazuh Integration

The Wazuh Agent was installed on pfSense through the FreeBSD package manager. FreeBSD repositories were temporarily enabled, the agent package was installed, and repositories were reverted to disabled to maintain pfSense stability. A symbolic link was created for the agent service, and it was enabled and started. A daily cron job was configured to clean logs older than 30 days.

pfSense syslog was configured in RFC 5424 format, and the Wazuh agent's ossec.conf was updated to monitor /var/log/filter.log. The Wazuh Manager was also configured with a remote syslog block to accept forwarded logs.



timestamp	agent.name	rule.description	rule.level	rule.id
Apr 3, 2025 @ 00:39:08.022	pfSense.home.arpa	Google IP Blocked	12	100101
Apr 3, 2025 @ 00:39:07.977	pfSense.home.arpa	Google IP Blocked	12	100101
Apr 3, 2025 @ 00:39:07.976	pfSense.home.arpa	Google IP Blocked	12	100101
Apr 3, 2025 @ 00:39:06.013	pfSense.home.arpa	Google IP Blocked	12	100101
Apr 3, 2025 @ 00:39:05.963	pfSense.home.arpa	Google IP Blocked	12	100101
Apr 3, 2025 @ 00:38:10.621	pfSense.home.arpa	pfSense Log arrived	5	100100
Apr 3, 2025 @ 00:38:10.620	pfSense.home.arpa	pfSense Log arrived	5	100100
Apr 3, 2025 @ 00:38:10.614	pfSense.home.arpa	pfSense Log arrived	5	100100
Apr 3, 2025 @ 00:38:10.614	pfSense.home.arpa	pfSense Log arrived	5	100100
Apr 3, 2025 @ 00:38:10.614	pfSense.home.arpa	pfSense Log arrived	5	100100
Apr 3, 2025 @ 00:38:10.564	pfSense.home.arpa	pfSense Log arrived	5	100100
Apr 3, 2025 @ 00:37:17.380	pfSense.home.arpa	pfSense Log arrived	5	100100
Apr 3, 2025 @ 00:37:17.371	pfSense.home.arpa	pfSense Log arrived	5	100100

7. Security Monitoring & Custom Rules

A custom decoder named 'pfsense' was created in local_decoder.xml using PCRE2 regex to extract fields from pfSense log entries including date, host, action, protocol, source IP, destination IP, and destination port. Custom rules were then created to detect and classify pfSense log events, including dedicated rules for blocked Google (Rule 100101, Level 12) and Facebook (Rule 100102, Level 12) traffic. After restarting the Wazuh Manager, real-time alerts appeared on the dashboard whenever blocked IPs were pinged from Kali.

8. Results

The Wazuh Dashboard displayed real-time alerts for both blocked IPs. Rule 100101 triggered for Google-bound traffic and Rule 100102 for Facebook-bound traffic, each generating Level 12 (high severity) alerts with full log context including source IP, destination, and action. This confirmed end-to-end integration from pfSense firewall event to Wazuh alert.

✕	2026-02-03 17:37:08.974308+05:00	WAN	Default deny rule IPv4 (1000000103)	0.0.0.0:68	255.255.255.255:67	UDP
✕	2026-02-03 17:37:10.988952+05:00	WAN	Block Google IP 142.250.190.78 (1769977093)	192.168.100.25	142.250.190.78	ICMP
✕	2026-02-03 17:37:12.006357+05:00	WAN	Block Google IP 142.250.190.78 (1769977093)	192.168.100.25	142.250.190.78	ICMP
✕	2026-02-03 17:37:13.029392+05:00	WAN	Block Google IP 142.250.190.78 (1769977093)	192.168.100.25	142.250.190.78	ICMP
✕	2026-02-03 17:37:14.083795+05:00	WAN	Block Google IP 142.250.190.78 (1769977093)	192.168.100.25	142.250.190.78	ICMP
✕	2026-02-03 17:37:15.103964+05:00	WAN	Block Google IP 142.250.190.78 (1769977093)	192.168.100.25	142.250.190.78	ICMP
✕	2026-02-03 17:37:16.104919+05:00	WAN	Block Google IP 142.250.190.78 (1769977093)	192.168.100.25	142.250.190.78	ICMP
✕	2026-02-03 17:37:17.105362+05:00	WAN	Block Google IP 142.250.190.78 (1769977093)	192.168.100.25	142.250.190.78	ICMP

Task 03 Malware Detection & Incident Response Plan

1. Objective

The objective of this task was to download a freely available malware test sample using a virtual machine, observe its detection through an active pfSense firewall configured with Squid Proxy and ClamAV, forward logs to Wazuh Manager, analyze the resulting logs, and develop a formal Incident Response Plan.

2. Lab Environment

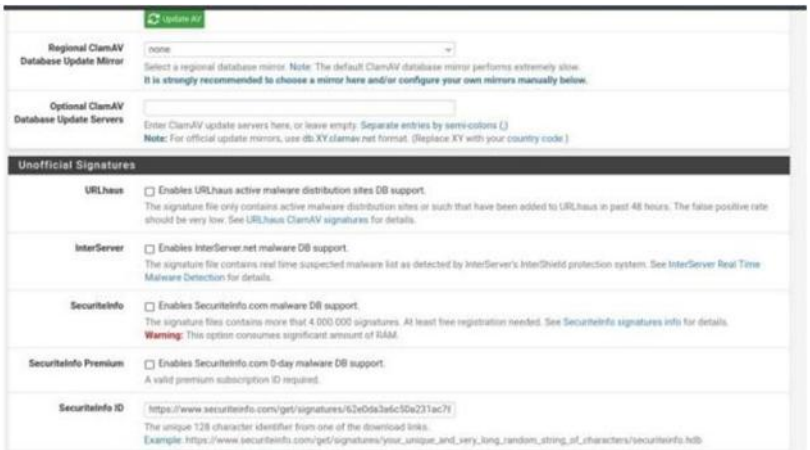
Three virtual machines were used: pfSense_Setup (firewall), main-ubuntu (Wazuh Manager), and Abdullah (Client VM running Lubuntu). Tools included pfSense Firewall, Squid Proxy, ClamAV Antivirus, Syslog-ng, Wazuh Manager, Oracle VirtualBox, and a standard web browser.

3. Setup & Configuration

The pfSense firewall was configured with a WAN interface (Bridged Adapter) and LAN interface (Internal Network). Squid Proxy was installed via the Package Manager with transparent HTTP proxy and SSL MITM filtering enabled. ClamAV was integrated for proxy-level antivirus scanning with Freshclam handling automatic signature updates. Syslog-ng was configured to forward firewall, Squid, and system logs to the Wazuh Manager at main-ubuntu on UDP port 5514.

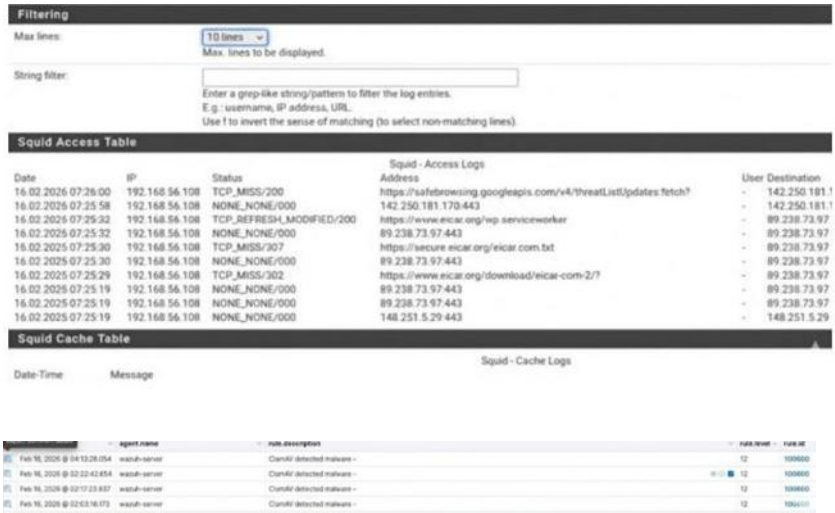
4. Malware Detection Test

To safely simulate malware activity, the EICAR test file was used — a universally recognized, harmless file designed specifically for testing antivirus detection. The client machine (Abdullah) attempted to download the EICAR file from https://www.eicar.org. The pfSense firewall, through ClamAV, detected the malware signature and blocked the request before it reached the endpoint. The browser displayed a virus warning, and a log entry was generated in pfSense under Status > System Logs > Proxy.



5. Log Forwarding to Wazuh

Wazuh Manager on main-ubuntu was configured to receive syslog logs by modifying /var/ossec/etc/ossec.conf with a remote configuration block. A custom decoder named clamav_detector was created to parse ClamAV log entries. A corresponding custom rule (Rule ID: 100600, Level 12 — High Severity) was written to trigger an alert whenever ClamAV detected malware. After restarting the Wazuh Manager, the alert appeared on the dashboard.



6. Incident Report Summary

Date: February 16, 2026 | Status: Closed

On February 16, 2026, a security incident involving an attempted malware download was detected in the controlled lab. The activity originated from client VM 'Abdullah'. The request was intercepted by pfSense_Setup, which detected the EICAR signature (Win.Test.EICAR_HDB-1) via ClamAV and blocked the download before it reached the endpoint. The full log entry read: Feb 16 07:25:31 pfSense_Setup VIRUS FOUND | Win.Test.EICAR_HDB-1 | https://secure.eicar.org/eicar.com.txt | 192.168.56.108.

7. Indicators Identified

Indicators of Compromise (IOCs): Malware Signature: Win.Test.EICAR_HDB-1, Malicious URL: https://secure.eicar.org/eicar.com.txt, Source IP: 192.168.56.108, Affected Machine: Abdullah.

Indicators of Attack (IOAs): The client initiated an HTTP GET request to a known malicious test file. Squid Proxy intercepted the traffic in transparent mode, ClamAV scanned content

in transit, the signature matched, and the download was blocked. No lateral movement was observed.

8. Incident Response Plan (NIST Framework)

The response followed the NIST Cybersecurity Framework: Identify → Protect → Detect → Respond → Recover.

- IR-1 Preparation: All systems were pre-configured (pfSense, Squid, ClamAV, Syslog-ng, Wazuh) with clearly defined roles.
- IR-2 Detection & Analysis: High-severity alert triggered in Wazuh. Analyst verified malware signature, source IP, and confirmed the event was isolated to the Abdullah VM.
- IR-3 Containment: Options included isolating the VM, blocking the malicious domain via firewall rules, and adding IOCs to blocklists. Increased log monitoring confirmed an isolated event with no lateral movement.
- IR-4 Eradication: Since malware was blocked before execution, no endpoint compromise occurred. Browser cache was cleared and a precautionary antivirus scan was performed with no malicious artifacts found.
- IR-5 Recovery: System integrity was reconfirmed, the VM was reconnected, and logs were monitored for 24 hours with no additional alerts.
- IR-6 Post-Incident Review: Detection timeline evaluated, improvements identified (stronger firewall policies, enhanced web filtering, refined Wazuh rules, potential EDR solution). Logs archived and threat intelligence database updated.

9. Conclusion

The attempted malware download was successfully detected, blocked, logged, and analyzed without endpoint compromise. The coordinated integration between pfSense_Setup, Squid Proxy, ClamAV, Syslog-ng, and Wazuh Manager demonstrated an effective layered security monitoring framework. The incident was formally closed as of February 16, 2026.

Task 04 Malware Analysis — Static, Dynamic & Hybrid

1. Objective

The goal of this task was to execute and analyze a malware sample in a controlled virtual environment using advanced tools, covering static analysis (examining the file without running it), dynamic analysis (observing live behavior), and a hybrid approach combining both. The ultimate deliverable was a comprehensive report comparing findings with existing threat intelligence.

2. Environment Setup

A sandbox environment was constructed using REMnux (a Linux-based malware analysis toolkit) and Windows 10 (victim machine for execution and monitoring), both running in Oracle VirtualBox with Host-Only network adapters. This isolated configuration ensures malware cannot communicate with the real network or host machine.

REMnux was configured with a static IP of 192.168.10.2 by editing the Netplan configuration file and running sudo netplan apply. Windows 10 was assigned a static IP of 192.168.10.3 with default gateway and DNS pointing to 192.168.10.2 (REMnux), enabling it to route all traffic through the analysis machine.

3. Malware Sample

A Snake Keylogger sample was downloaded from MalwareBazaar (bazaar.abuse.ch) using its SHA256 hash: 4b3fccf8e3e8cf25b5b63ba8f397687dd733fd884efe7c78083d154310f272e4. The file was transferred to REMnux via a VirtualBox Shared Folder.

4. Static Malware Analysis

Static analysis examines a file's properties, code structure, strings, and metadata without executing it. The following steps were performed on REMnux:

- The file command confirmed the sample is a PE32 executable (32-bit Windows format).
- pecheck extracted metadata including MD5/SHA hashes, detected high entropy in the .rsrc section (nearly 8/8), indicating packed or encrypted content, and revealed key imported libraries: USER32.dll (keylogging), WSOCK32.dll (network communication), and ADVAPI32.dll (registry persistence).
- ExeInfo PE on Windows identified the compiler as AutoIt3 Scripting Language (v3.3.3.16), indicating the attacker scripted the keylogger using AutoIt.
- Ghidra was used for disassembly and decompilation. Analysis of the Symbol Tree revealed suspicious DLL imports: ADVAPI32.DLL for registry manipulation and service creation (persistence), USER32.DLL for keylogging and input capture, and various file system functions (DeleteFileA, WriteFile, CreateFile) suggesting evidence destruction and data exfiltration capabilities.
- String analysis in Ghidra revealed HTTP strings confirming the malware communicates with a remote server to send stolen data. The .ndata PE section and functions like GetTickCount and Sleep suggest anti-analysis techniques.

5. Dynamic Malware Analysis

Dynamic analysis involves executing the malware in a controlled environment and observing its behavior in real-time. INetSim was launched on REMnux to simulate a fake network (HTTP, DNS, SMTP, FTP, and other services), allowing the malware to interact with what it believes are real servers while remaining safely contained.

On Windows, Process Monitor (Procmon) and Process Explorer from Microsoft Sysinternals were used to monitor system activity. After executing the malware, the following behaviors were observed:

- Process Hijacking: The malware injected itself into RegSvcs.exe (a legitimate Windows process) to avoid detection.
- System Fingerprinting: It queried the Windows Registry for the computer name and hardware identifiers.
- VM Detection: It examined video and graphics settings to detect virtual machine environments.
- Keylogging Preparation: It loaded system DLLs required for capturing keyboard input and credentials.

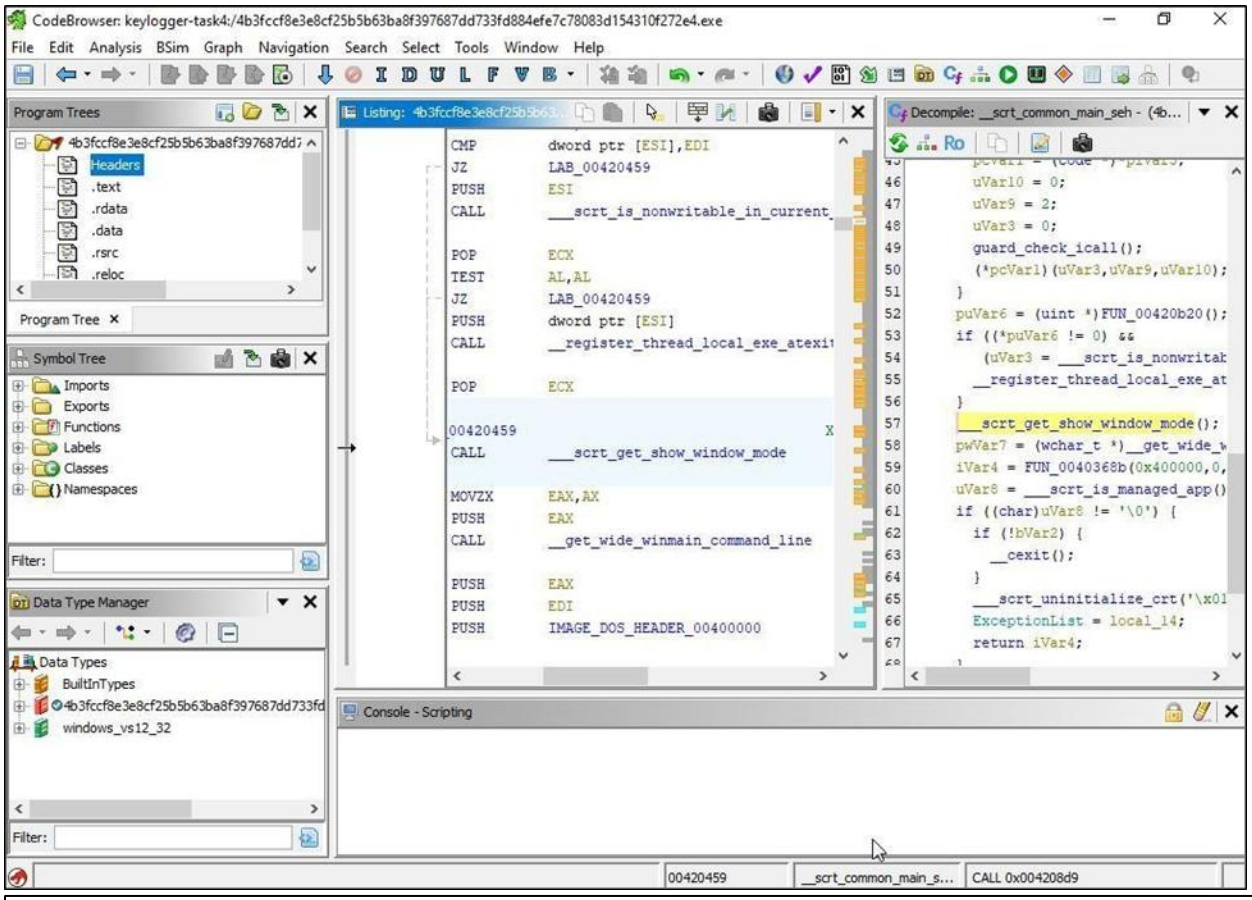
6. Hybrid Malware Analysis

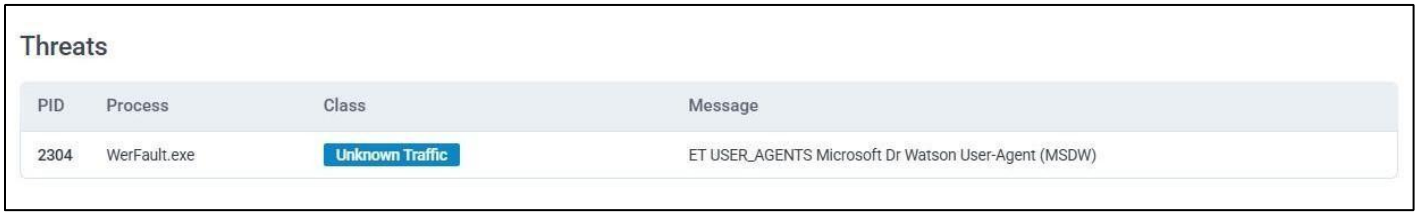
Hybrid analysis combines both static and dynamic approaches. The sample was submitted to ANY.RUN, an interactive online sandbox, for a combined analysis session. The process tree showed: malicious .exe → unbraricading.exe → regsvcs.exe → werfault.exe, with werfault.exe used to disguise behavior.

MITRE ATT&CK mapping from ANY.RUN confirmed: Persistence (T1547.001) via Startup folder, and Discovery (T1012) through extensive registry queries. Key IOCs identified included unique file hashes, the VBS persistence script (unbarricading.vbs), and suspicious network traffic masquerading as Microsoft Dr. Watson communications — indicating an attempt to exfiltrate stolen data to a Command & Control (C2) server.

7. Incident Response Plan for Snake Keylogger

6. Detection: Check for suspicious processes (unbarricading.exe, abnormal RegSvcs.exe), monitor registry for mass read activity, and search for known IOCs (MD5 hash, unbarricading.vbs in Startup folder).
7. Containment: Disconnect the infected machine from the internet to block C2 communication. Disable the startup script and terminate suspicious processes using Task Manager.
8. Eradication: Delete malicious files from AppData\LocalLow and Temp folders, remove harmful registry entries, and perform a full system scan with Microsoft Defender or Malwarebytes.
9. Recovery: Change all important passwords from a clean device (since this is a keylogger). Restore to a pre-infection restore point if available. Apply the latest Windows updates and security patches.





Threats

PID	Process	Class	Message
2304	WerFault.exe	Unknown Traffic	ET USER_AGENTS Microsoft Dr Watson User-Agent (MSDW)

Task 05 Wazuh & n8n Integration — Automated SOC Alert Pipeline

1. Task Overview

This task involved integrating Wazuh SIEM with the n8n workflow automation platform using webhooks, and implementing an automated SOC Alert Aggregation and Reporting Pipeline as the unique use case.

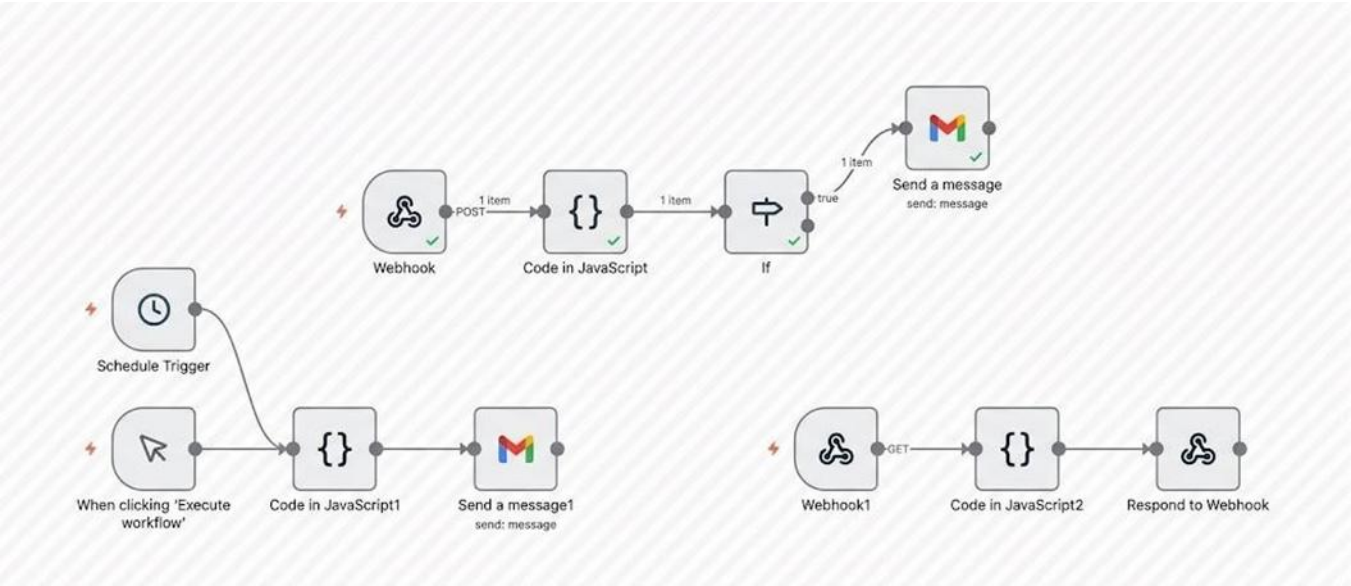
2. Objective

Design and implement an automated pipeline that: receives all real-time security alerts from Wazuh via Webhook; automatically classifies each alert by severity (Critical / High / Medium / Low); fires an immediate email notification for critical alerts (level 10+); generates a scheduled daily digest email with severity breakdown and trend analysis; and provides a live browser-accessible dashboard with a real-time severity chart and recent alerts table.

3. Use Case Design — Three-Branch Architecture

The workflow operates across three parallel branches:

- Alert Intake: Wazuh sends POST alerts to the webhook. The Code node classifies alerts by severity and stores them. An IF node routes critical alerts (level 10+) to an immediate Gmail notification.
- Daily Digest: A Schedule Trigger fires at midnight daily. A Code node counts stored alerts, calculates trends vs. the previous run, identifies the most triggered rule, and sends a formatted HTML digest email.
- Live Dashboard: A GET webhook serves an HTML page with Chart.js bar chart showing severity distribution, summary cards (Total/Critical/High/Medium/Low), and a recent alerts table — all rendered fresh on every browser request.



4. Infrastructure & Tools

- Wazuh Server: CentOS VM — IP 192.168.168.10, connected via VMware VMnet2
- Host Machine: Windows Laptop — IP 192.168.168.1 on VMnet2
- n8n: Installed on Windows Host, accessible on port 5678
- Gmail OAuth2 via Google Cloud Console for email notifications

5. Network Configuration

The CentOS VM was on VMnet2 but initially the host had no virtual adapter. This was resolved by opening VMware Workstation > Edit > Virtual Network Editor, selecting VMnet2, and enabling 'Connect a host virtual adapter to this network'. The host was then manually assigned IP 192.168.168.1/24. Connectivity was verified from both sides with ping.

A Windows Firewall inbound rule was created for TCP port 5678 to allow the CentOS VM to reach n8n. n8n was started with N8N_HOST=0.0.0.0 to listen on all interfaces rather than only localhost.

6. Wazuh Integration

The Wazuh ossec.conf was edited to add a custom integration block pointing to a script named custom-n8n. The integration script at /var/ossec/integrations/custom-n8n used curl to forward alert JSON payloads via POST to the n8n webhook URL (http://192.168.168.1:5678/webhook/wazuh-alerts). SSH brute force attempts were generated from the Windows host against the CentOS VM to trigger real Wazuh alerts, producing Rule IDs 5710, 5712, 5760, and 5762 at levels 5, 7, 9, and 10.

Export Formatted | Reset view | 637 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Mar 30, 2026 @ 20:32:04.5...	wazuh-soc-agent	sshd: connection reset	4	5762
Mar 30, 2026 @ 20:32:04.5...	wazuh-soc-agent	sshd: Attempt to login using a non-existent user	5	5710
Mar 30, 2026 @ 20:32:04.5...	wazuh-soc-agent	sshd: Attempt to login using a non-existent user	5	5710
Mar 30, 2026 @ 20:32:04.5...	wazuh-soc-agent	sshd: brute force trying to get access to the system. Non existent user.	10	5712
Mar 30, 2026 @ 20:32:02.5...	wazuh-soc-agent	sshd: Attempt to login using a non-existent user	5	5710
Mar 30, 2026 @ 20:31:44.5...	wazuh-soc-agent	sshd: Attempt to login using a non-existent user	5	5710
Mar 30, 2026 @ 20:31:44.5...	wazuh-soc-agent	sshd: Attempt to login using a non-existent user	5	5710
Mar 30, 2026 @ 20:31:44.5...	wazuh-soc-agent	sshd: connection reset	4	5762
Mar 30, 2026 @ 20:31:42.5...	wazuh-soc-agent	sshd: Attempt to login using a non-existent user	5	5710
Mar 30, 2026 @ 20:31:42.5...	wazuh-soc-agent	sshd: Attempt to login using a non-existent user	5	5710
Mar 30, 2026 @ 20:09:15.1...	wazuh-soc-agent	SCA summary: CIS CentOS Linux 7 Benchmark v3.1.2.: Score less than 50% (35)	7	19004
Mar 30, 2026 @ 20:09:06.8...	wazuh-soc-agent	CIS CentOS Linux 7 Benchmark v3.1.2.: Ensure nonessential services are removed or masked.: Status changed fro...	3	19010

7. Gmail OAuth2 Setup

A Google Cloud project (n8n-wazuh) was created with the Gmail API enabled. An OAuth 2.0 Client ID (Web Application type) was configured with the n8n redirect URI. The credential was connected in n8n by entering the Client ID and Secret, then completing the Google sign-in flow. This enabled the Gmail nodes to send authenticated emails.

8. Testing & Results

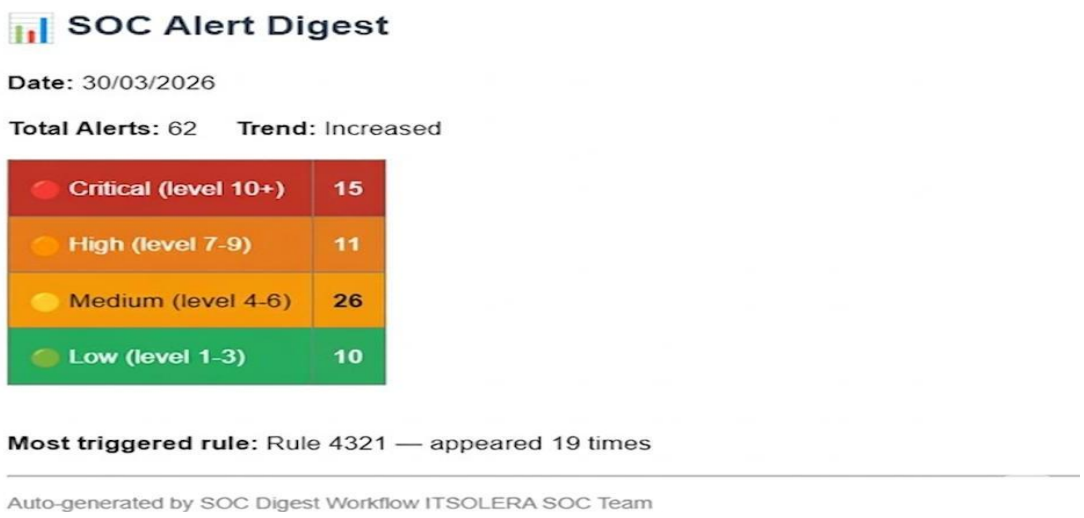
The workflow was first tested using PowerShell to simulate fake alert payloads. Real Wazuh alerts were then generated by running SSH brute force attempts. The n8n workflow executed successfully: critical alerts (level 10) triggered immediate email notifications, the

daily digest email showed a full severity breakdown, and the live dashboard updated in real-time with rule IDs, descriptions, agents, and color-coded severity levels.

9. Troubleshooting Log

Key issues resolved during this task:

- Host couldn't ping CentOS VM: VMnet2 had no host adapter — resolved by enabling it in VMware Virtual Network Editor.
- VMnet2 received 169.254.x.x IP: No static IP on Windows — resolved by manually setting 192.168.168.1.
- curl from CentOS couldn't reach n8n: Windows Firewall blocking port 5678 — resolved by adding an inbound rule.
- curl hung with no response: n8n was listening on localhost only — resolved by starting n8n with N8N_HOST=0.0.0.0.
- Email fields showing empty: JSON structure mismatch between test and real Wazuh alerts — resolved by updating the Code node to handle both structures.
- Gmail OAuth Error 403: Gmail address not added as a test user in Google Cloud Console — resolved by adding it to the OAuth consent screen.



10. Conclusion

This implementation demonstrates real-world SOC automation at three levels: real-time critical escalation, scheduled batch reporting, and live visual monitoring. When Wazuh detects security events, they are automatically classified by severity, stored, and made available through both email notifications and a browser-based dashboard — entirely without manual intervention. The daily digest with trend analysis mirrors reporting found in enterprise SIEM platforms such as Splunk and IBM QRadar, making this a practically relevant and professionally grounded implementation.